Sistemi Operativi
Da Unix a GNU/Linux

Docente: Claudio E. Palazzi

cpalazzi@math.unipd.it

Gestione dei processi (UNIX/Linux) – 1

- **Processo**
  - La principale entità attiva nel sistema
  - Concorrenza a livello di processi
    - Molti processi attivati direttamente dal sistema (*daemon*)
      - appuntamenti
    - Creazione mediante `fork()`
      - Clone con stessa memoria all'inizio e accesso a file aperti
    - Comunicazione mediante scambio messaggi (*pipe*) e invio di segnali (*signal*) entro un gruppo
  - Processi figli hanno memoria identica quella del processo genitore <u>solo all'inizio</u>
    - Poi **indipendente** alla prima modifica
    - <u>Copy-on-write</u>

Gestione dei processi (UNIX/Linux) – 2

- Processi con più flussi di controllo interni
  - Detti *thread*
  - La creazione di un *thread* gli assegna identità, attributi, compito e argomenti

```
res = pthread_create( &tid , attr , fun , arg )
```

  - Thread <u>condividono tutte le risorse</u> logiche e fisiche del processo genitore (inclusi valori di variabili in uso)

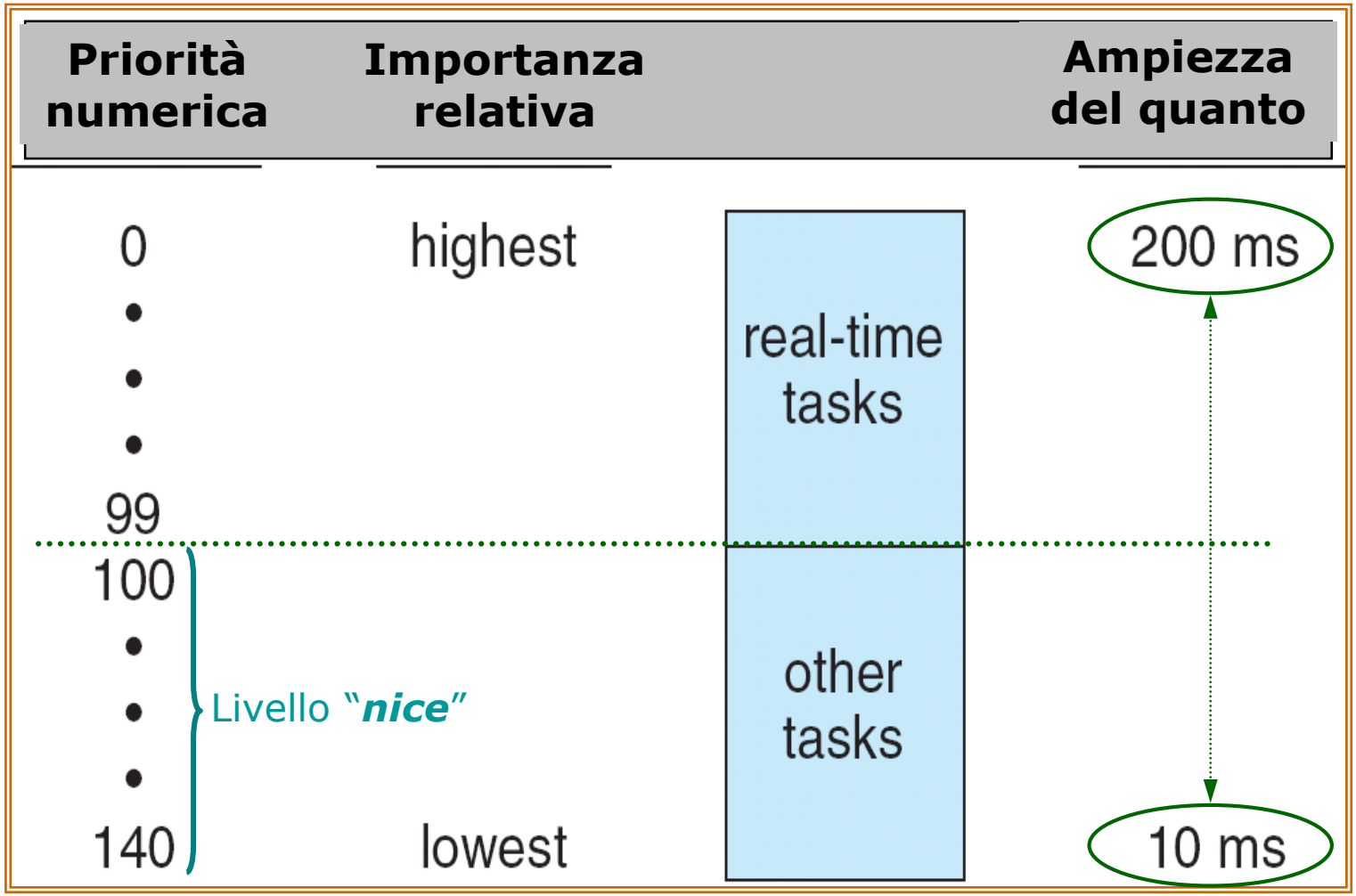
Gestione dei processi (UNIX/Linux) – 3

- `fork()` duplica il processo chiamante creando un processo figlio uguale ma distinto
  - Che accade se questi include più *thread*?
- Vi sono 2 possibilità
  - **Tutti i *thread* del padre vengono clonati**
    - Difficile gestire il loro accesso concorrente ai dati e alle risorse condivise con *thread* del padre
  - **Solo un *thread* del padre viene clonato**
    - Possibile sorgente di inconsistenza rispetto alle esigenze di cooperazione con le *thread* non clonate
- Dunque il *multi-threading* aggiunge gradi di complessità
  - Al FS
    - Più difficile assicurare consistenza nell'uso concorrente di *file*
  - Alla comunicazione tra entità attive
    - Come decidere il *thread* destinazione di un segnale inviato a un processo?
    - Un thread era bloccato da lettura da tastiera, i nuovi thread anche? E chi riceverà i caratteri digitati?

Ordinamento dei processi (GNU/Linux) – 1

- I *thread* sono gestiti direttamente dal nucleo
  - Ordinamento per *task* (*thread* o processo indistintamente)
  - Selezione distinta tra classi distinte
  - Prerilascio per fine quanto o per attesa di evento
- 3 classi di priorità di *task*
  - **Tempo reale con politica FCFS a priorità senza prerilascio**
    - A priorità uguale viene scelto il *task* in attesa da più tempo
  - **Tempo reale con politica RR a priorità**
    - Prerilascio per quanti con ritorno in fondo alla coda
  - **Divisione di tempo RR a priorità (*Timesharing*)**
    - Priorità dinamica con premio o penalità rispetto al grado di interattività con I/O (alta → premio, bassa → penalità)
    - Nuovo valore assegnato al *task* all'esaurimento del suo quanto corrente

Ordinamento dei processi (GNU/Linux) – 2



Silberschatz, Galvin and Gagne ©2005

Gestione della memoria (UNIX)

- In origine l'allocazione di memoria principale avveniva mediante ***swap*** di processi
  - Rimpiazzo di interi processi quando una particolare esecuzione rilevava mancanza di memoria
  - Il gestore (***swapper***) creava lo spazio necessario salvando su disco i processi sospesi con più tempo d'esecuzione recente e minor priorità
- In seguito, fu introdotta paginazione con modalità **a richiesta** (***paging on demand***)
  - Nessun caricamento anticipato di pagine

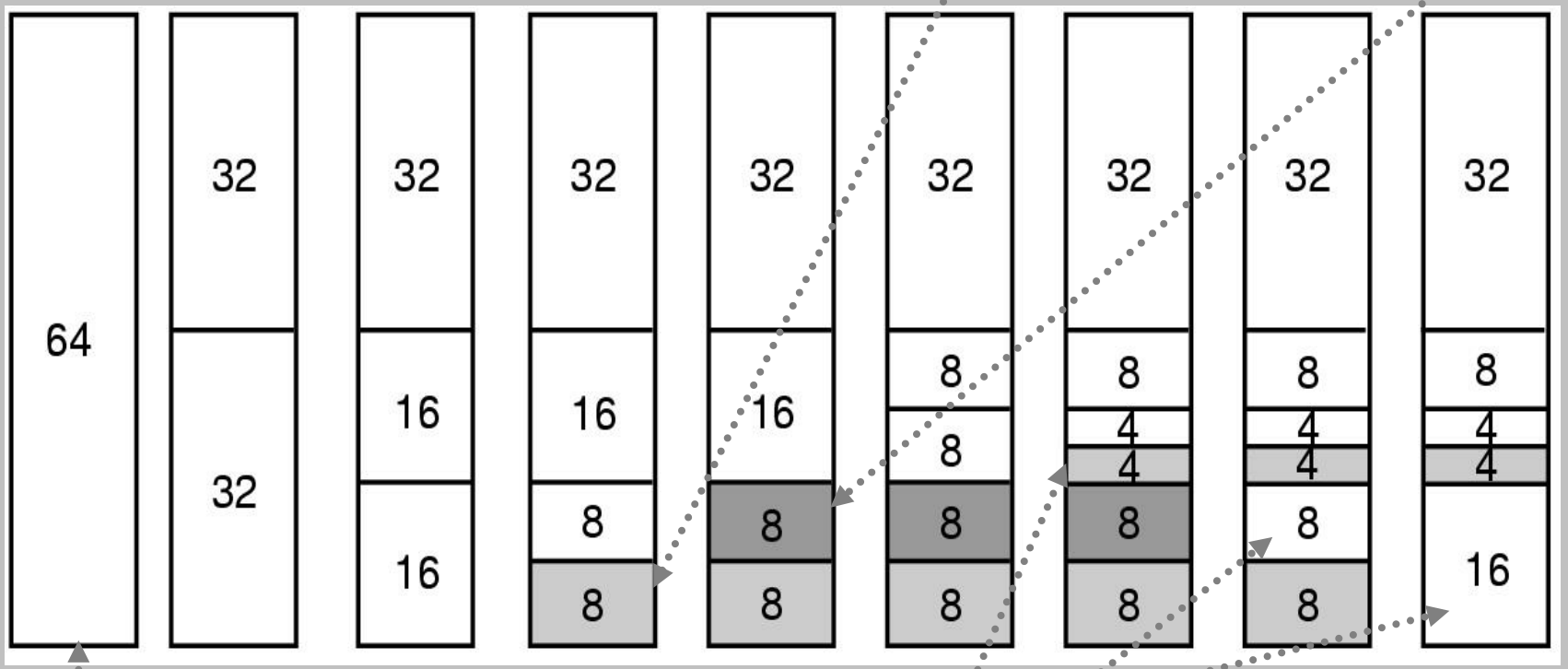
Gestione della memoria (GNU/Linux)

- **La RAM è allocata in maniera dinamica e variabile**
- **Algoritmo di allocazione primario** (*buddy*)
  - Ogni richiesta di ampiezza N è arrotondata a $2^m \geq N$
  - La memoria disponibile viene frazionata in metà successive fino a frazioni di ampiezza 2^m
    - Una singola frazione viene assegnata al richiedente
    - Una struttura ausiliaria contiene la testa di liste predefinite di frazioni di ampiezza 2^i (i=0, …, m) per velocizzare la ricerca
  - La memoria disponibile usata per l'allocazione è sempre la frazione libera di minore dimensione
  - Al rilascio ogni frazione tornata libera si unisce con la frazione vicina se libera (il suo *buddy*)
  - algoritmi sussidiari cercano di ridurre la frammentazione causata dall'algoritmo primario
    - Strutture ***slab*** per evitare di usare 128 pagine per allocarne 65

Funzionamento del *buddy algorithm*

3. Richiesta di **5** pagine → allocazione di 2^3=**8**

2. Richiesta di **6** pagine contigue → allocazione di 2^3=**8**



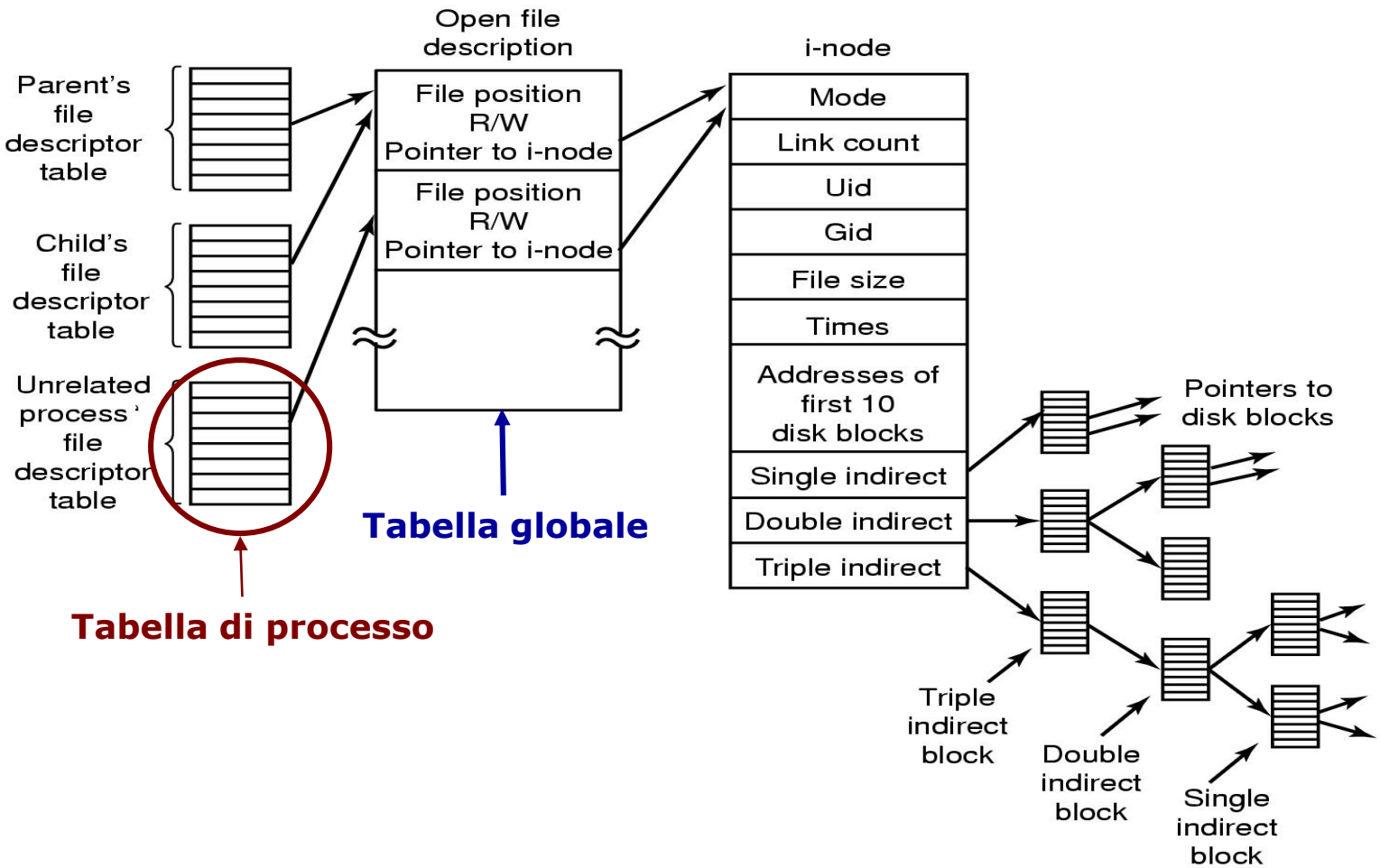
4. Richiesta di **4** pagine → allocazione di 2^2=**4**

5-6. Rilascio e ricompattazione

1. Inizialmente un unico insieme contiguo di 64 pagine

Dimensione min e max area allocabile?

Realizzazione del FS in UNIX – 3



Realizzazione del FS in UNIX – 4

- L'*i-node* principale del *file* contiene (tra l'altro) l'indirizzo dei suoi primi 12 blocchi dati
  - 1 *i-node* ha la dimensione di 1 frazione di blocco (64 B)
- Per *file* più grandi 1 campo dell'*i-node* principale punta a 1 *i-node* secondario che contiene puntatori ad altri blocchi dati
  - I-*node* principale con campo *single-indirect*
- Per *file* ancora più grandi l'*i-node* secondario contiene puntatori a nodi *single-indirect*
  - I-*node* principale con campo *double-indirect*
- È previsto anche un campo *triple-indirect*

Esempio d'uso di *i-node* (UNIX v7)

- **Ipotesi**
  - Blocco dati di capienza 4 KB
  - *i-node* ampio 64 B
  - Indici di blocco espressi su 4 B
- **Esempio 1** (con uso di campo *single-indirect*)
  - Max dimensione di *file* rappresentabile
    - $(12 + 64\ B / 4\ B) \times 4\ KB = (12 + 16) \times 4\ KB = 112\ KB$
- **Esempio 2** (con uso di campo *double-indirect*)
  - Max dimensione di *file* rappresentabile
    - $112\ KB + 16^2 \times 4\ KB = 1\ MB + 112\ KB$
- **Esempio 3** (con uso di campo *triple-indirect*)
  - Max dimensione di *file* rappresentabile
    - $1\ MB + 112\ KB + 16^3 \times 4\ KB = 17\ MB + 112\ KB$

Realizzazione del FS GNU/Linux – 1

- Inizialmente basato sul FS di MINIX però subito abbandonato per le eccessive limitazioni
  - Limitazioni imposte da MINIX
    - Nomi ≤ 14 caratteri
    - Indirizzi di blocco su 2 B per blocchi ampi 1 KB
    - Ampiezza massima di file ≤ 64 MB (**perché?**)
- `ext2` diviene presto la versione di riferimento
  - Basata sulle scelte BSD con diversa struttura fisica
  - La maggiore innovazione è stata la suddivisione della partizione in **gruppi di blocchi**
    - Distribuzione uniforme delle directory su disco
    - *i-node* e relativi blocchi dati sono tenuti **vicini** tramite preallocazione di alcuni blocchi al momento della creazione di un file
      - Riduce frammentazione

Realizzazione del FS GNU/Linux – 3

- Dimensione di *i-node* estesa a 128 B
  - Indirizzi di blocco ampi 4 B
    - Per denotare fino a 2^{32} = 4 G blocchi
  - Blocchi di dimensione 1, 2, 4 KB scelta in fase di configurazione del FS
    - Partizione di dimensione ≥ 4 TB
    - 12 indirizzi diretti + 3 indiretti (*single*, *double*, *triple*)
    - Informazioni di controllo
    - Una parte riservata per uso futuro
- Ogni aggiunta a *file* viene realizzata quanto più **localmente** possibile entro lo stesso gruppo
  - Località **tra** *file* correlati tramite gruppi
  - Località **entro** *file* mediante preallocazione di N ≤ 8 blocchi contigui